

Be proactive, take responsibility, think from the user's perspective, make thoughtful decisions, and finish what you start.

Ownership: "I took responsibility," "I didn't wait to be asked," "I owned it end-to-end," "I fixed the root cause."

Dive Deep: "I traced the issue," "I reproduced the bug," "I checked logs/metrics," "I isolated the bottleneck," "I found the root cause."

Deliver Results: "I prioritized the critical path," "I removed blockers," "I delivered within the deadline," "the result was X% improvement."

Bias for Action: "I had incomplete information," "I made a reversible decision," "I validated it quickly," "I took a calculated risk."

Learn and Be Curious: "I hadn't used this before," "I learned it quickly," "I built a prototype," "I applied it directly to the project."

Customer Obsession: "user pain point," "reduced friction," "improved the experience," "worked backward from the user."

Invent and Simplify: "automated the process," "removed unnecessary steps," "simplified the workflow," "made it reusable."

Earn Trust: "I listened to their concern," "I acknowledged my mistake," "I communicated clearly," "I used data instead of opinion."

Disagree and Commit: "I respectfully disagreed," "I presented evidence," "we discussed tradeoffs," "once we decided, I fully supported it."

Highest Standards: "temporary workaround," "permanent fix," "edge cases," "prevented recurrence," "improved reliability."

Are Right, A Lot: "I validated my assumptions," "considered alternatives," "changed my mind based on evidence."

Think Big: "designed for future scale," "made it reusable," "looked beyond the immediate requirement."

Frugality: "used existing resources," "reduced infrastructure," "optimized CPU/memory," "avoided unnecessary cost."

1. Ownership - Tell me about a time you took ownership of something that wasn't explicitly your responsibility. (Code Journey Development)

Situation:

While working as a TA during my master's, I noticed it was difficult to tell whether students were writing their code themselves or simply pasting large sections into the editor.

Task:

This wasn't part of my assigned TA responsibilities, but I felt it affected how fairly the professor could evaluate the work, so I decided to build a solution.

Action:

I developed a platform that tracked coding behavior such as keystrokes, cursor movement, deletions, edits, and timing patterns. Naturally written code usually showed gradual typing and frequent corrections, while large paste events created a very different activity pattern. The system flagged suspicious sessions for professor review rather than making the final decision automatically.

Result:

It gave the professor an additional way to evaluate how the code was produced, and it solved a problem I identified without being asked. It also taught me to look beyond my assigned responsibilities when I see a problem I can improve.

2. Ownership - Tell me about a time you noticed a problem and fixed it without being asked. (FileKit – Squash compression)

Situation:

I created an open-source file management system that included features like file organization, metadata handling, and file compression. While testing the compression module, I noticed that my initial approach using Huffman encoding with byte packing was reducing file sizes by only around 30% on average.

Task:

Improving the compression ratio wasn't something anyone specifically asked me to do, but I felt that 30% wasn't good enough for the product, so I decided to investigate it further.

Action:

I analyzed the limitation and realized Huffman encoding was good at reducing symbol-level redundancy, but it wasn't taking advantage of repeated byte sequences in the file. I researched different compression techniques and integrated LZ77 before the Huffman encoding stage so repeated sequences could first be represented more efficiently.

Result:

That improved the average compression from around 30% to more than 50%, with some files

achieving even better results. It was a good example of noticing a weakness in something I had already built and proactively improving it without being asked.

3. Dive Deep - Tell me about a difficult bug or production issue you investigated. (DeepFake Analysis Image Upload Bottleneck)

Situation:

As part of my research work, I developed an image-analysis model using DlibCNN, VGG16, and Transformers. The model worked seamlessly in my local environment, but after deploying it to production, larger images created a bottleneck because we initially had only a single processing instance.

Task:

I needed to understand why the same model that worked well locally was slowing down in production and find a way to process larger images without overwhelming the server.

Action:

I traced the issue to the size of the image payloads and the way they were being sent and processed.

I first applied lossless compression locally to reduce the payload size without affecting image quality. For larger images, I split the data into smaller chunks and sent those chunks independently to the server.

On the server side, I associated the chunks with the request/thread identifier, reconstructed the original image once all chunks arrived, and then passed the reconstructed image to the model for inference.

Result:

This removed the large-image transfer bottleneck and allowed the production system to process larger images more reliably without requiring changes to the underlying ML model.

The experience taught me to separate model performance from the complete production pipeline, because a system that works locally can still fail because of networking, payload size, or infrastructure constraints.

4. Dive Deep - Tell me about a time when the data/metrics did not match what you expected.

Situation:

During my research on fake-news detection, I initially experimented with traditional machine-learning models, but the evaluation metrics were consistently below what I expected.

Task:

I needed to understand whether the problem was coming from the model itself or from the quality of the text data.

Action:

I analyzed the dataset and found a lot of unnecessary and noisy text. I iteratively cleaned the data

by removing irrelevant content and normalizing the text, then retrained and compared the results after each change.

Even after improving the dataset, the traditional ML models were limited in capturing the context and sequence of words, so I moved to a CNN-BiLSTM architecture, where CNN extracted local textual features and BiLSTM captured context in both directions.

Result:

The deep-learning approach produced significantly better evaluation metrics and became the architecture I used for the final research model. The experience taught me not to treat a bad metric as just a model problem—I first validate the data, preprocessing, and assumptions behind the metric.

5. Deliver Results - Tell me about a project where you faced major obstacles but still delivered. (Reason behind EdFlix Portal & HLS protocol)

Situation:

While pursuing my master's, I was also teaching students through E-ClassRoom an online meeting platform built using Stream IO API. As the number of sessions grew, live teaching became difficult to scale alongside my coursework.

Task:

I decided to build **EdFlix**, a structured on-demand learning platform, instead of relying only on live classes or public video uploads.

Action:

I built the platform using React and Spring Boot. A major obstacle was buffering and network latency for users with slower connections. I traced the issue to serving large video files directly and redesigned the streaming layer using **HLS with M3U8 playlists and smaller video segments**.

Result:

I launched the platform successfully, it grew to **500+ daily users**, and buffering dropped by around **40%**. It allowed me to continue delivering course content reliably while managing my academic workload.

6. Deliver Results - Tell me about a time you had a difficult deadline. (Code journey story, shipped core features first then later released UI/UX updates)

Situation:

During my master's, I was balancing coursework, TA responsibilities, grading, and two research papers. At the same time, I was asked to deliver a student code-analysis tool that could help identify whether code was typed naturally or pasted.

Task:

I had a limited window to build and deliver the tool without affecting my academic work or TA responsibilities.

Action:

I broke the work into small milestones and focused first on the minimum useful functionality: capturing keystrokes, cursor movement, edits, and paste-related timing patterns. I prioritized the core detection logic first, then added the reporting layer for the professor. I also blocked fixed time for grading, coursework, and research so none of the responsibilities slipped.

Result:

I delivered the analyzer within the required timeline while continuing to manage my TA work and research commitments. The experience taught me how to prioritize aggressively and still deliver a usable solution under multiple competing deadlines.

7. Failure - Tell me about your biggest failure. (Re-architecting instead of Re-cleaning data)

Situation:

During my fake-news detection research, the model metrics were below what I expected. My initial reaction was to spend significant time redesigning and experimenting with different model architectures.

Task:

My goal was to improve the model performance enough to produce reliable research results.

Action:

After multiple iterations, I realized I had focused too much on changing the architecture and not enough on understanding the underlying dataset. I went back, analyzed the data more deeply, removed noisy and irrelevant samples, improved preprocessing, and only then finalized the CNN-BiLSTM architecture.

Result:

The model performance improved, but my biggest learning was that I had spent unnecessary time optimizing the architecture before validating the data properly. Since then, I first validate the data, assumptions, and bottleneck before making major architectural changes.

8. Failure / Earn Trust - Tell me about a mistake you made and how you handled it. (Code journey initial version)

Situation:

While building the student code analyzer, my first version relied heavily on sudden typing and cursor-pattern changes to identify possible copy-paste activity.

Task:

During testing, I realized that this approach could incorrectly flag legitimate behavior, which was a serious problem because the tool was being used to support academic evaluation.

Action:

I took responsibility for the design and changed the system so it no longer relied on a single signal. I combined multiple factors such as typing patterns, cursor movement, edits, timing, and paste

events, and made the system flag suspicious sessions for professor review instead of making an automatic decision.

Result:

The revised approach was more reliable and reduced the risk of false conclusions. The main lesson for me was that when a system can affect people, I need to design for false positives and edge cases from the beginning, not just the happy path.

9. Failure - Tell me about a time you made the wrong technical decision. (Deployment of image analyzer on monolithic arch, instead of micro-services based)

Situation:

As a Research Assistant, I deployed an image-analysis model using a monolithic architecture. It worked initially, but as concurrent requests increased, heavy inference requests started slowing down the entire application.

Task:

I realized my original architecture was limiting scalability and needed to be corrected.

Action:

I redesigned the system into microservices, separating preprocessing, inference, and supporting services so each component could scale independently.

Result:

The new design handled higher load more reliably and made scaling much more efficient. My key learning was to choose architecture based on expected workload and scaling needs, not just initial simplicity.

10. Learn and Be Curious - Tell me about a time you had to learn a new technology quickly. (Learning electron JS, to build users trust)

Situation:

While building FileKit, I first developed the core file-management functionality and exposed it through a web interface for easy access.

Task:

I wanted the application to feel more like a native desktop product, work completely offline, and provide better access to local files and system resources.

Action:

I had not worked deeply with Electron.js before, so I learned it quickly and integrated it with my existing Java and React stack. I also learned how to use IPC communication between the UI and backend processes and handled platform-specific behavior across Windows, Linux, and macOS.

Result:

I converted FileKit into a cross-platform desktop application that could run fully offline with a native experience while keeping the same core functionality. The experience taught me to learn

technologies based on the product problem I'm trying to solve, rather than limiting myself to the stack I already know.

11. Bias for Action - Tell me about a time you had incomplete information but had to make a decision. (3D to 2D dataset, for SAM2-Unet)

Situation:

During one of my research projects, I was asked to work with a SAM2-UNet model designed for 2D image segmentation, but the assigned BraTS dataset contained 3D MRI volumes.

Task:

There wasn't a predefined pipeline for converting the 3D data into a format the model could consume, and I had to decide how to proceed without losing important tumor information.

Action:

I analyzed the MRI volumes and decided to convert them into 2D slices, while prioritizing slices containing larger tumor regions instead of treating every slice equally. I then created the preprocessing pipeline and used those selected 2D segments for training and evaluation.

Result:

This allowed me to adapt the 3D dataset to the 2D SAM2-UNet architecture and continue the research without waiting for a completely new model or dataset. The main lesson was to make a reasonable technical decision with the information available, validate it quickly, and refine it based on results.

12. Customer Obsession - Tell me about a time you improved something for a customer/user. (Made marks calculation easy for students)

Situation:

During my undergraduate studies at JNTUH, I noticed that students often struggled to calculate their GPA manually because the grading structure, credits, and subject combinations made the process confusing.

Task:

I wanted to make the process simpler so students could quickly understand their academic performance without repeatedly calculating everything by hand.

Action:

I built a JNTUH GPA calculator where students could enter their grades and credits, and the application automatically calculated the GPA based on the university grading rules. I focused on keeping the interface simple so students could use it without understanding the calculation logic behind it.

Result:

It made GPA calculation much easier and reduced the manual effort for students. The experience taught me to first understand where users are struggling and then build the simplest solution that directly removes that friction.

13. Invent and Simplify - Tell me about something you automated or simplified. (Attendance management story)

Situation:

While working with students, attendance tracking and manually notifying absent students was repetitive and time-consuming.

Task:

I wanted to automate the process so instructors didn't have to continuously calculate absences and send individual reminders.

Action:

I built an attendance management system that automatically tracked each student's absences, sent an email whenever a student was absent, and monitored their overall attendance. If a student's attendance dropped below the required threshold, the system automatically notified the professor.

Result:

This reduced manual tracking and follow-up work and gave both students and professors timely visibility into attendance issues.

14. Earn Trust - Tell me about a conflict with a teammate. (Normal streaming VS HLS)

Situation: While building EdFlix, a teammate wanted to keep direct video streaming because it was simpler, but I was concerned about buffering on slower networks.

Task: We needed to decide whether the simpler approach was good enough for production.

Action: I tested both approaches under different network conditions and showed that direct streaming degraded significantly. I proposed HLS with M3U8 segments, and after discussing the tradeoffs, we agreed to switch.

Result: Buffering dropped by about 40%, and I learned to resolve technical disagreements with data rather than opinion.

15. Have Backbone; Disagree and Commit -Tell me about a time you disagreed with your manager/team. (Deployment on monolithic arch vs micro-services based)

Situation: While working as a Research Assistant, our image-analysis application was deployed as a monolith. The team preferred keeping it that way because it was simpler to maintain.

Task: As concurrent inference requests increased, I felt scaling the entire application for one compute-heavy component was inefficient.

Action: I raised the concern with the team and backed it with load-test results. I proposed separating preprocessing and model inference into independent services so we could scale only the bottleneck. After discussing the added complexity and tradeoffs, the team agreed to the change.

Result: We moved to a microservice-based architecture, which improved scalability and resource utilization. I learned that when I disagree, I should bring data and alternatives—not just an opinion.

16. Highest Standards - Tell me about a defect/problem you fixed permanently rather than applying a temporary solution.

Situation: At ITTacademy, some APIs were slow because background work like email notifications was being executed synchronously.

Task: A temporary fix would have been increasing timeouts or adding more server resources, but that wouldn't solve the root cause.

Action: I redesigned the flow by separating non-critical work from the request path and introduced an asynchronous, thread-pooled background processing pipeline.

Result: API wait time dropped by about 70%, and the system could handle background jobs much more efficiently. The key lesson was to fix the architecture causing the problem instead of masking the symptom.

17. Are Right, A Lot - Tell me about a difficult technical decision you made. (Applying lossless compression and chunking image, for deep-fake analysis)

Situation: While deploying my image-analysis model, larger images created a bottleneck in production because the system was running on a limited number of instances.

Task: I had to decide whether to simply scale the infrastructure or redesign how image data was transferred and processed.

Action: I chose to first optimize the pipeline. I applied lossless compression, split large images into smaller chunks, sent them independently, and reconstructed them on the server using the request identifier before inference.

Result: This reduced the transfer bottleneck and allowed the existing infrastructure to handle larger images more reliably. It taught me to optimize the system bottleneck before solving every performance problem by adding more infrastructure.

18. Feedback - Tell me about critical feedback you received and what you changed.

Situation: During my fake-news detection research, I was spending a lot of time experimenting with different model architectures because the metrics were below expectations.

Task: My professor gave me critical feedback that I was focusing too much on the model and not enough on understanding the dataset itself.

Action: I took the feedback seriously and changed my approach. I analyzed the data more deeply, removed noisy and irrelevant samples, improved preprocessing, and only then continued model experimentation with CNN-BiLSTM.

Result: The evaluation results improved, and I became much more systematic in how I approach technical problems. Now I validate the data and assumptions before making major architectural changes.

19. Ownership - Tell me about a time you had to deliver something with very little guidance. (OS Wizard story)

Situation:

I was asked to build **OS Wizard**, a web application covering 10 operating-system problems, but I was given almost no guidance on the UI, UX, or how the simulations should work.

Task:

I had to decide how to present complex OS computations in a way that students could easily follow.

Action:

I designed the interface myself and broke each problem into step-by-step visual simulations. I focused on showing how inputs changed, how each computation progressed, and why each result was produced instead of just displaying the final answer.

Result:

I delivered a usable learning tool that made OS concepts easier for students to understand. The experience taught me how to take a vague requirement, define the product behavior myself, and deliver without needing detailed instructions.

20. Dive Deep - Tell me about a problem where the obvious solution turned out to be wrong. (Adding more servers VS pre-processing image & chunking)

Situation:

While deploying my image-analysis model, larger images caused significant latency in production.

Task:

The obvious solution was to add more server instances, but before scaling infrastructure, I wanted to confirm the actual bottleneck.

Action:

I traced the request flow and found that much of the delay came from transferring large image payloads rather than model inference itself. Instead of simply adding servers, I applied **lossless**

compression, split larger images into chunks, and reconstructed them on the server before inference.

Result:

The pipeline handled larger images more reliably without immediately increasing infrastructure. I learned to validate the root cause before applying the most obvious solution.

21. Deliver Results - Tell me about a time you failed to meet a goal. What happened? (GARUDA story)

Situation:

Out of personal interest, I started building reusable Java APIs and eventually wanted to simplify the instruction model further by creating my own language processor, **GARUDA**.

Task:

My goal was to build a proper compiler pipeline with a parser. I implemented memory management and core data structures in C, bridged them with Java, and translated instructions into C.

Action:

To make progress quickly, I initially processed the language using regular expressions. That worked for the basic instruction set, but as the language became more complex, the regex-based approach became difficult to maintain. I had planned to replace it with a proper parser, but I underestimated the scope and didn't complete that part within the time I had available.

Result:

I ended up with a working language processor, but I did not achieve my original goal of building a full parser-based compiler. The experience taught me to define smaller milestones early and validate the compiler architecture before expanding the language.

22. Learn and Be Curious - Tell me about something you taught yourself that helped your project. (Learnt electron.js to give UI/UX for my offline apps)

Situation:

While building FileKit, I had already developed the core file-management functionality and a web interface, but I wanted the application to work fully offline and feel like a native desktop product.

Task:

I had never worked deeply with **Electron.js**, so I needed to learn it quickly and integrate it with my existing Java and React application.

Action:

I taught myself Electron, learned IPC communication between processes, and worked through platform-specific file-system behavior across Windows, Linux, and macOS.

Result:

I converted FileKit into a cross-platform desktop application with native-like offline access. It taught me how to independently learn a new technology and apply it directly to a real product need.

23. Learn and Be Curious - Tell me about a time you received feedback and changed your approach. (Research story, Arch vs Data cleansing)

Situation: During my fake-news detection research, I was spending a lot of time experimenting with different model architectures because the metrics were below expectations.

Task: My professor pointed out that I was focusing too much on the model and not enough on understanding the underlying data.

Action: I changed my approach and first analyzed the dataset, removed noisy and irrelevant samples, improved preprocessing, and then evaluated which architecture actually fit the data best.

Result: The model performance improved, and since then I've followed a **data-first, root-cause-first approach** before making major architectural changes.

24. Bias for Action - Tell me about a calculated risk you took. (Risk of losing data while converting from 3D to 2D)

Situation:

During my research, I was asked to use a 2D SAM2-UNet model with the 3D BraTS MRI dataset.

Task:

There was no predefined conversion pipeline, and simply slicing the volume randomly could lose important tumor information.

Action:

I took a calculated approach: I converted the 3D volumes into 2D slices, but prioritized slices containing larger tumor regions. I validated the transformed data through experiments before committing to the full training pipeline.

Result:

The approach let me continue using the assigned 2D model without switching datasets or

redesigning the entire architecture. It taught me to take risks only after identifying the downside and building a way to validate the decision quickly.

25. Bias for Action - Tell me about a time you had to act quickly to solve an urgent problem. (Incorrect floating point operations on JNTUH app)

Situation: I built an Android GPA calculator for students, but after release I found that I had not handled floating-point division correctly, which caused inaccurate results in some cases.

Task: I needed to give students a correct solution immediately, but publishing an Android update would take time because of the app-store review process.

Action: I fixed the calculation logic right away and quickly deployed a web version so students could continue using the calculator correctly while the mobile update was under review.

Result: Students had an immediate working alternative, and I released the corrected Android version about a week later. It taught me that during an urgent issue, I should first restore a reliable user experience, then follow through with the permanent fix.

26. Customer Obsession - Tell me about a difficult customer requirement and how you handled it. (PDF/Data offline caching in MRIT app, for improper net connection)

Situation: Students wanted real-time college news and academic updates, but many also had inconsistent internet connectivity.

Task: I needed to make sure the app stayed useful even when users were offline or on a weak network.

Action: I combined Firebase Realtime Database and REST APIs for live updates, added push notifications for important announcements, and implemented offline caching so previously fetched content was still available without connectivity.

Result: The app served **1,200+ active users** and improved accessibility and engagement by about **80%**. It taught me to design around actual user constraints, not just ideal conditions.

27. Customer Obsession - Tell me about a time you went beyond the original requirement because it improved the user experience. (Added push notifications & messaging service)

Situation:

The original requirement for the college app was simply to provide students with news and academic updates.

Task:

While using the app, I realized students could still miss important updates when they had poor connectivity or weren't actively checking the application.

Action:

Beyond the original requirement, I added **push notifications and offline caching** so important

announcements reached students proactively and previously loaded content remained available without internet access.

Result:

The app became much more reliable and convenient for students, eventually serving **1,200+ active users** and improving accessibility and engagement by about **80%**.

28. Invent and Simplify - Tell me about a time you came up with a new solution to an existing problem. (Digital finger print for registered devices for EdFlix)

Situation:

While building EdFlix subscriptions, I wanted to prevent one account from being freely shared across multiple devices.

Task:

In a browser-based application, I couldn't reliably access a device's physical hardware identifier, so I needed another way to recognize a registered device.

Action:

I created a lightweight device fingerprint using browser information, screen characteristics, and a generated unique token. I stored that association with the user account and validated it during login before allowing access.

Result:

This gave me a practical way to limit account sharing without requiring native hardware access or additional software installation. It taught me to work within platform constraints and design a simpler alternative when the ideal information isn't available.

29. Earn Trust - Tell me about a time you had to convince your team to accept your idea. (Deployment on Micro-services based arch)

Situation:

As a Research Assistant, our image-analysis system was initially deployed as a monolith. The team preferred keeping it because it was simpler to maintain.

Task:

As inference traffic increased, I felt scaling the whole application for one compute-heavy component was inefficient, so I proposed separating the workload.

Action:

I ran load tests, identified inference as the main bottleneck, and showed the team how independent services would let us scale preprocessing and inference separately. I also explained the tradeoff: slightly more operational complexity in exchange for better scalability and isolation.

Result:

The team agreed to move toward a microservice-based design, which improved scalability and resource utilization. I learned that the best way to influence a technical decision is to bring data, tradeoffs, and a practical migration path.

30. Disagree and Commit - Tell me about a decision you disagreed with but supported after the decision was made. (Working with 3D based model instead of State of the art SAM2-Unet)**Situation:**

During my research, we were asked to use a **2D SAM2-Unet model** with the **3D BraTS MRI dataset**. I initially felt a 3D-based approach would preserve the volumetric information better.

Task:

I raised my concern, but the project direction was to continue with SAM2-Unet.

Action:

Once the decision was made, I committed to it and focused on making the approach work. I converted the 3D MRI volumes into 2D slices and prioritized slices containing larger tumor regions so we retained the most useful information for segmentation.

Result:

We were able to successfully use the assigned model with the BraTS dataset and complete the research work. I learned that after respectfully raising a concern, I need to fully support the agreed direction and make it successful.

31. Frugality - Tell me about a time you achieved something despite limited resources. (Pre-processing image and chunking, DeepFake analysis)**Situation:**

While building ITTAcademy, I had very limited budget and infrastructure, but I still wanted to provide students with a complete learning ecosystem.

Task:

I needed to support live classes, recorded content, and collaborative coding without relying on expensive third-party platforms.

Action:

I built the core products myself: an **online classroom**, a **GitHub-like code repository with web**

and CLI access, and **EdFlix** for on-demand learning. I reused common backend components, open-source technologies, and shared infrastructure instead of purchasing separate services for each requirement.

Result:

I was able to build and operate a complete learning ecosystem with very little spend while still supporting real students and multiple use cases. It taught me to solve resource constraints through reuse and good architecture before adding cost.

32. Failure - Tell me about a mistake that affected other people.

33. Failure - Tell me about a project that didn't go as planned.

34. Reflection - Tell me about something you would do differently if you could redo a project.
(GARUDA re-start story)

Situation:

While building my GARUDA language processor, I initially used regular expressions to parse instructions because it allowed me to build the first version quickly.

Task:

As I added more instructions, the parsing logic became increasingly difficult to maintain and extend.

Action:

If I could redo the project, I would define the language grammar first and build a proper tokenizer and parser from the beginning instead of relying heavily on regex.

Result:

That would have required more upfront effort, but it would have made the language much easier to extend, debug, and maintain. The project taught me to think about long-term extensibility before choosing the fastest initial implementation.

Usage of LLMs as an SDE

1. What is an LLM?

An LLM is a neural network trained on large amounts of text to predict the next token. Through that training, it learns language patterns and can perform tasks like summarization, coding, question answering, and generation.

2. What is a token?

A token is a small unit of text processed by an LLM. It may be a whole word, part of a word, punctuation, or a symbol.

3. What is a context window?

The context window is the maximum amount of text or tokens the model can consider in one request, including the prompt, conversation history, and generated response.

4. What is hallucination?

Hallucination is when an LLM generates information that sounds reasonable but is factually incorrect or unsupported.

To reduce it, I can use:

RAG, trusted sources, structured prompts, validation, and human review for critical decisions.

5. What is RAG?

Retrieval-Augmented Generation retrieves relevant information from an external knowledge source and provides it to the LLM as context before generating the answer.

Example:

User question → embedding → retrieve relevant company documents → send documents + question to LLM → generate answer.

6. Why use RAG?

I would use RAG when the model needs access to **private, frequently changing, or domain-specific information**.

It also helps reduce hallucinations by grounding responses in retrieved content.

7. What are embeddings?

Embeddings are numerical vector representations of text that capture semantic meaning.

Texts with similar meanings generally have vectors that are closer together.

8. What is a vector database?

A vector database stores embeddings and supports similarity search.

In a RAG system, I can convert the user's question into an embedding and retrieve the most semantically similar documents.

9. RAG vs fine-tuning?

RAG changes what information the model receives.

Fine-tuning changes the model's behavior or learned patterns.

I would use RAG for changing/private knowledge and fine-tuning when I want consistent behavior, style, or task specialization.

10. How would you build an LLM system that answers questions from company documents?

I would:

Documents → clean/chunk → generate embeddings → store in vector DB → embed user query → retrieve relevant chunks → send chunks + query to LLM → return answer with sources.

That's a basic RAG architecture.

11. How would you reduce hallucinations?

I would:

- ground answers using RAG
- provide clear instructions
- restrict the model to trusted context
- ask it to say when information is unavailable
- validate important outputs
- include source citations where possible

12. How would you evaluate an LLM application?

I wouldn't evaluate only whether the response “sounds good.”

I would measure things like:

accuracy, relevance, groundedness, hallucination rate, latency, cost, and user satisfaction.

For RAG, I'd separately evaluate retrieval quality and generation quality.

13. What is temperature?

Temperature controls randomness in generation.

Lower temperature: more deterministic and consistent.

Higher temperature: more diverse and creative.

For something like code generation or factual Q&A, I would generally prefer a lower temperature.

14. How would you reduce LLM latency?

I could:

- use a smaller/faster model where appropriate
- reduce unnecessary prompt/context size
- retrieve fewer but more relevant documents
- cache repeated requests
- stream responses
- parallelize independent operations

15. How would you reduce LLM cost?

I would:

- use smaller models for simpler tasks
- reduce token usage
- cache common results
- summarize long conversation history
- retrieve only relevant context
- batch requests when appropriate
- use larger models only for harder tasks

16. How would you protect sensitive data when using an LLM?

I would avoid unnecessarily sending sensitive information to the model.

I would use:

access controls, data minimization, encryption, redaction, secure providers, logging controls, and clear retention policies.

17. What happens if the prompt exceeds the context window?

Some content must be removed, summarized, or retrieved dynamically.

For a large document system, I would generally use **chunking + retrieval** instead of sending every document to the LLM.

18. What is prompt engineering?

Prompt engineering means structuring instructions and context so the model produces more reliable output.

For example, instead of:

“Analyze this.”

I could say:

“Analyze this error log, identify the likely root cause, list supporting evidence, and suggest three fixes ordered by risk.”

19. LLM vs traditional ML?

Traditional ML is usually trained for a specific task, such as classification.

An LLM is a general-purpose language model capable of performing many tasks from instructions.

If I need a small, predictable classification system with low latency, traditional ML may still be the better choice.

20. What is an AI agent?

An LLM normally generates a response.

An **agent** can use the LLM to reason about a task and then interact with tools or APIs to take actions.

For example:

User: “Schedule a meeting.”

An agent might inspect calendars, find availability, and call a calendar API.

21. What if the LLM gives different answers to the same question?

LLMs are probabilistic, so some variation is expected.

If consistency matters, I can:

- lower temperature
- use stronger system instructions
- provide structured context
- constrain output format
- validate results programmatically

22. How would you make an LLM answer only using trusted information?

I would use RAG with trusted sources and instruct the model:

“Answer only using the provided context. If the answer isn't present, say you don't have enough information.”

For critical applications, I'd also validate citations or retrieved evidence.

23. When would you NOT use an LLM?

I wouldn't use an LLM when:

- deterministic logic is sufficient
- exact mathematical correctness is required
- latency/cost constraints are very strict
- a simple database lookup solves the problem
- incorrect output could cause serious harm without validation

A good engineer shouldn't use an LLM just because it's available.

24. What are the main limitations of LLMs?

Main limitations include:

hallucination, limited context, nondeterminism, outdated/missing knowledge, cost, latency, privacy concerns, and inability to guarantee correctness.

25. How do you use LLMs in your daily software-development work?

A good answer for **you**:

“I use LLMs mainly as a productivity multiplier. I use them to understand unfamiliar APIs, summarize documentation, generate initial test cases, review code for edge cases, and brainstorm debugging approaches. I also use them to improve technical documentation. But I don't treat their output as automatically correct I verify important technical decisions through documentation, tests, and my own reasoning.”